

前言

对于一个标准的 前端工程化 项目，开发与测试两不误。可能有同学会说，不是还有 QA 吗，何必浪费时间搞测试？QA 是为应用的 测试环境 与 生产环境 而服务，偏向测试应用的界面与功能，结合交互文档与设计文档找出应用的 Bug。

应用由依赖、类库、模块、构建代码、业务代码等代码块组成，有第三方封装的代码块，也有自己封装的代码块。在 QA 看来，他们可不会区分得这么细，应用出问题就是出问题，管你在哪里报错，通通提单。

他们的工作任务更多是找出应用在线上可能出现的交互问题，至于底层代码可能导致的 Bug，他们可能无法直接测试出来，只有当底层代码的逻辑运行到某种特殊条件才可能触发，例如 边界、空值、溢出、多重条件组合 等情况。

这些问题很难通过测试用例模拟出来，因此更多时候是在开发阶段找出这些因为 边界、空值、溢出、多重条件组合 等情况导致的问题，这样才能保障应用在生产环境中稳定运行。

针对这些底层代码，一般都是自己封装的类库，有条件能在开发阶段接入测试工具走一遍测试流程，保障代码块的稳定性与准确性，是一件很棒的事情，绝对能让你负责的应用在生产环境中少一份担忧。本章将带领你**基于Jest为类库编写测试用例**，通过一步一步上手 `jest`，为工具库添加 单元测试 与 代码覆盖测试，保障高质量的代码设计与高质量的代码。

背景：单元测试是什么

测试代码块的可行性通常都使用 单元测试。单元测试指检查与验证软件中最小可测试单元。对于 单元测试 中单元定义，一般来说要根据实际情况判定其具体含义。

例如 JS 中单元指一个函数，Java 中单元指一个类，GUI 中单元指一个 窗口 等。简而言之，单元就是人为规定最小的被测功能模块。单元测试是软件开发时要进行的最低级别测试活动，独立单元将在与程序其他部分隔离的情况下进行测试。

近几年前端技术高速发展，系统功能变得越来越复杂，这对 前端工程化 的能力提出更高要求，听到工程化的第一反应必然是高质量的代码设计与高质量的代码实现。如何确保这些环节的稳定，那 单元测试 就必须得应用起来了。

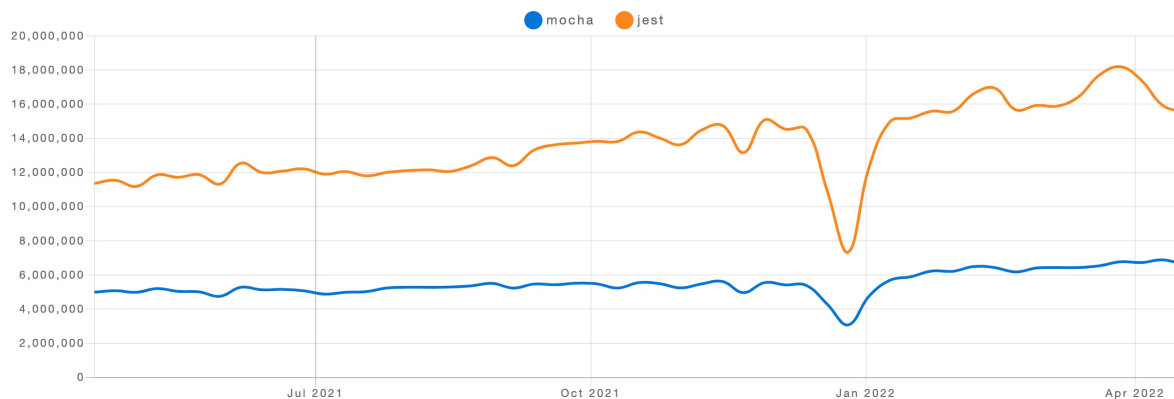
单元测试 的四大特性就完美诠释了其重要作用。



像一些大型前端项目，例如 `react`、`vue`、`babel`、`webpack` 等都会接入 单元测试，可见 单元测试 对于这些明星项目来说是多重要的。因为这些大型前端项目需处理大规模的产品及其频繁迭代的功能，这种可持续化的迭代方式迫使它们必须引入 自动化测试。进一步看，单元测试 有助于增强整体质量与减少运维成本。

目前主流的 前端单元测试框架 主要有 `jest` 与 `mocha`，但更推荐使用 `jest`，因为 `jest` 与 `mocha` 从多方面对比都有更明显的优势。

Downloads in past 1 Year



Stats

			Stars	Issues	Version	Updated	Created	Size
	mocha	  	21,240	263	9.2.2	a month ago	10 years ago	 minizipped size 63.5 KB
	jest	  	38,648	1,081	27.5.1	3 months ago	10 years ago	 bundlephobia  timeslip  掘金技术社区

jest 作为一款优秀的 前端单元测试框架，有着简单易用、高性能、易配置和多功能的特性，内置监控、断言、快照、模拟、覆盖率等功能，这些功能都能开箱即用，因此是作为类库测试的首选工具。

expect与test

单元测试 有两个很重要的概念函数，分别是 **expect()** 与 **test()**。**expect()** 表示期望得到的运行结果，简称期望结果；**test()** 表示测试结果是否通过预期，简称通过状态。以下从一个简单示例了解这两个函数的工作原理。

声明一个简单的功能函数 **Sum()**，用于累加入参。

```
function Sum(...vals) {  
    return vals.reduce((t, v) => t + v, 0);  
}  
  
Sum(1, 2, 3); // 6
```

js

若 **Sum()** 不小心写错把累加写成累乘，当直接在业务代码中使用该函数就会带来无法预期的 **Bug**，需对该函数进行 自动化测试，确保无问题再打包为 **bundle**文件 供业务组件调用，这样就保障该函数的准确性。

执行以下代码若发现未报错，则表示代码运行结果符合预期。这就是 自动化测试 最原始的雏形，是一个最简单的 单元测试 的测试用例。

js

```
const result = Sum(1, 2, 3);
const expect = 6;

if (result !== expect) {
    throw new Error(`期望是${expect}, 结果是${result}`);
}
```

把代码改造下变成以下形式，是不是感觉更简洁？

js

```
expect(Sum(1, 2, 3)).toBe(6); // 是否等于9
expect(Sum(1, 2, 3)).toBe(9); // 是否等于10
```

`expect()` 的实现原理也很简单，入参一个值再返回一个 `toBe()`，当然也可返回更多自定义的期望函数。

js

```
function expect(result) {
    return {
        toBe(expect) {
            if (result !== expect) {
                throw new Error(`期望是${expect}, 结果是${result}`);
            }
        }
    };
}
```

运行上述两行 `expect().toBe()` 代码，结果第二行代码报错。虽然实现了 `expect()`，但报错内容始终一样，不知具体哪个函数出现问题。为了强化 `expect()` 的功能，需对其做进一步改良，若在 `expect()` 外部再包装一层函数就可传递更多信息进来。

js

```
test("期望结果是6", () => {
    expect(Sum(1, 2, 3)).toBe(6);
});
test("期望结果是9", () => {
    expect(Sum(1, 2, 3)).toBe(9);
});
```

上述封装既能得到运行结果是否符合期望，又能得到详细的自定义测试描述。

```
function test(desc, fn) {  
  try {  
    fn && fn();  
    console.log(`${desc} 通过测试`);  
  } catch {  
    console.log(`${desc} 未通过测试`);  
  }  
}
```

自动化测试

从上述分析可知，**自动化测试** 其实就是编写一些测试函数，通过测试函数运行业务代码，判断实际结果与期望结果是否相符，是则通过，否则不通过，整个过程都是通过预设脚本 **自动化** 处理。

上述 `expect()` 与 `test()` 都是主流的 **前端单元测试框架** 的内置函数，其语法完全一样，它们在不同 **前端单元测试框架** 中的实现原理也大同小异。

方案：基于Jest为类库编写测试用例

以第15章开发的工具库为例，通过一步一步上手 **jest**，为 **bruce-us** 的每个工具函数加入 **单元测试** 的测试用例。

安装

因为使用 **typescript** 编码，所以安装 **jest** 时也需把 **typescript** 相关依赖一起安装。

```
npm i -D @types/jest jest ts-jest
```

在根目录中创建 **jest.config.js** 文件，用于配置 **jest** 测试配置。**jest** 整体配置简洁明了，可查看[Jest官网](#)，主要用到的配置选项包括 **preset** 与 **testEnvironment**。
preset 表示预设模板，使用安装好的 **ts-jest** 模板；**testEnvironment** 表示测试环境，可选 **web/node**。

ts-jest 为 **jest** 与 **typescript** 环境中的 **单元测试** 提供类型检查支持与预处理。

```
module.exports = {
  preset: "ts-jest",
  testEnvironment: "node"
};
```

js

在 `package.json` 中指定 `scripts`，加入测试命令。 `--no-cache` 表示每次启动测试脚本不使用缓存； `--watchAll` 表示监听所有 单元测试，若发生更新则重新执行脚本。

```
{
  "scripts": {
    "test": "jest --no-cache --watchAll"
  }
}
```

json

在 `tsconfig.json` 中指定 `types`，加入 `@types/jest`。 `@types/jest` 提供了 `expect()` 与 `test()`。

```
{
  "compilerOptions": {
    "types": [
      "@types/jest"
    ]
  },
  "exclude": [
    "jest.config.js"
  ]
}
```

json

单元测试

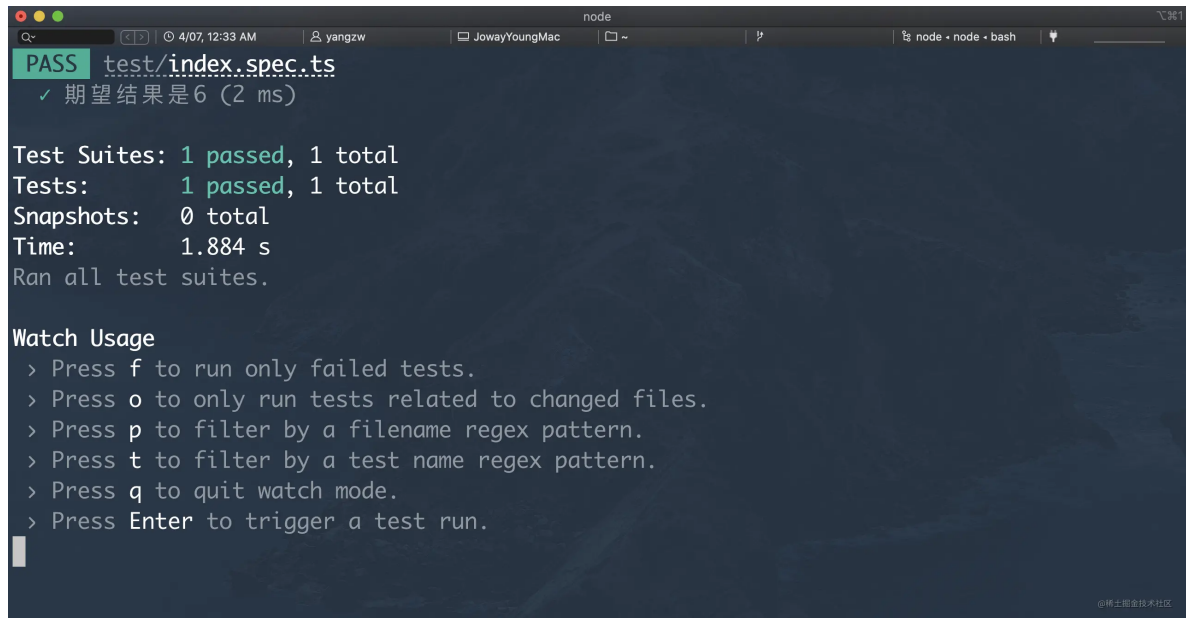
在根目录中创建 `test` 文件夹。 `test` 文件夹内部的目录结构可参照 `src` 文件夹，保持源码与测试脚本的目录结构一样，方便后续迭代与维护。 单元测试 的测试用例使用 `xyz.spec.ts` 的方式命名。因为文件众多，那些非重要的文件就不展示了。另外 `src` 文件夹中的 `index.ts`、`node.ts` 和 `web.ts` 三种文件是供 `rollup` 识别入口，因此无需加入对应测试用例文件。

```
| |─ common
| | |─ array.ts
| | |─ boolean.ts
| | |─ date.ts
| | |─ function.ts
| | |─ number.ts
| | |─ object.ts
| | |─ regexp.ts
| | |─ string.ts
| |─ node
| | |─ fs.ts
| | |─ os.ts
| | |─ process.ts
| | |─ type.ts
| |─ web
| | |─ cookie.ts
| | |─ dom.ts
| | |─ function.ts
| | |─ storage.ts
| | |─ type.ts
| | |─ url.ts
|─ test
| |─ common
| | |─ array.spec.ts
| | |─ boolean.spec.ts
| | |─ date.spec.ts
| | |─ function.spec.ts
| | |─ number.spec.ts
| | |─ object.spec.ts
| | |─ regexp.spec.ts
| | |─ string.spec.ts
| |─ node
| | |─ fs.spec.ts
| | |─ os.spec.ts
| | |─ process.spec.ts
| | |─ type.spec.ts
| |─ web
| | |─ cookie.spec.ts
| | |─ dom.spec.ts
| | |─ function.spec.ts
| | |─ storage.spec.ts
| | |─ type.spec.ts
| | |─ url.spec.ts
```

先将上述 `Sum()` 走一次测试用例。在 `test` 文件夹中创建 `index.spec.ts` 文件，加入以下内容。打开 **CMD工具**，执行 `npm run test`，输出以下信息表示测试通过。

ts

```
function Sum(...vals: number[]): number {  
    return vals.reduce((t, v) => t + v, 0);  
}  
  
test("期望结果是6", () => {  
    expect(Sum(1, 2, 3)).toBe(6);  
});
```



```
node  
Q~ 4/07, 12:33 AM yangzw JowayYoungMac ~ node • node • bash  
PASS test/index.spec.ts  
✓ 期望结果是6 (2 ms)  
  
Test Suites: 1 passed, 1 total  
Tests: 1 passed, 1 total  
Snapshots: 0 total  
Time: 1.884 s  
Ran all test suites.  
  
Watch Usage  
  > Press f to run only failed tests.  
  > Press o to only run tests related to changed files.  
  > Press p to filter by a filename regex pattern.  
  > Press t to filter by a test name regex pattern.  
  > Press q to quit watch mode.  
  > Press Enter to trigger a test run.
```

加入以下代码，测试脚本会重新执行，输出以下信息表示测试不通过。第二段 `expect().toBe()` 测试不通过，期待结果是 9，运行结果是 6。当修正期待结果就会自动通过测试了。

ts

```
test("期望结果是9", () => {  
    expect(Sum(1, 2, 3)).toBe(9);  
});
```



```
node
4/07, 12:33 AM  yangzw  JowayYoungMac  node • node • bash
FAIL test/index.spec.ts
✓ 期望结果是6 (2 ms)
✗ 期望结果是9 (2 ms)

● 期望结果是9

expect(received).toBe(expected) // Object.is equality

Expected: 9
Received: 6

66 | });
67 | test("期望结果是9", () => {
> 68 |     expect(Sum(1, 2, 3)).toBe(9);
    |                               ^
69 | });

at Object.<anonymous> (test/index.spec.ts:68:23)
```

通过上述操作说明 `jest` 已被部署到 `bruce-us` 中了。因为工具函数众多，因此选取一个工具函数为例，后续有时间可一起完善 `bruce-us` 的测试用例。

代码覆盖测试

上述 `单元测试` 只不过是验证运行结果是否符合预期，着重于结果。若产生一个运行结果的过程可能存在多个分支，例如以下加强版的 `Sum()`，总共有3个分支，`Sum(1, 2, 3)` 只满足其中一个分支，验证完毕也只占整个函数的 `33.33%`，剩下 `66.66%` 的代码还未能验证，那就不能百分百认为该 `单元测试` 成功通过。

```
function Sum(...vals: number[]): number {
    if (vals.length === 0) {
        return -1;
    } else if (vals.length === 1) {
        return 0;
    } else {
        return vals.reduce((t, v) => t + v, 0);
    }
}
```

若要将整个 `Sum()` 验证完毕，必须将所有 `if-else` 语句验证一遍，当全部满足条件才能百分百认为该 `单元测试` 成功通过。这就引入一个测试概念，`代码覆盖测试`。

代码覆盖测试指程序源码被测试的比例与程度的所得比例。`代码覆盖测试` 生成的指标称为 `代码覆盖率`，它作为一个指导性指标，可在一定程度上反应测试的完备程度，是软件质量度的重要手段。`100%` 覆盖率的代码并不意味着 `100%` 无 Bug，代

码覆盖率 作为质量目标无任何意义，应把它作为一种发现未被测试覆盖的代码的检查手段。简而言之， 代码覆盖测试 更注重测试过程，而测试结果只是测试过程的一个表现。

jest 本身内置 代码覆盖测试，改良上述配置就能运行 代码覆盖测试 了。修改 `jest.config.js` 的配置，追加 `coverage` 相关配置。修改 `package.json` 中 `scripts` 的 `test`，追加 `--coverage`。

```
module.exports = {  
  coverageDirectory: "coverage",  
  coverageProvider: "v8",  
  coverageThreshold: {  
    global: {  
      branches: 100,  
      functions: 100,  
      lines: 100,  
      statements: 100  
    }  
  },  
  preset: "ts-jest",  
  testEnvironment: "node"  
};
```

js

```
{  
  "scripts": {  
    "test": "jest --no-cache --watchAll --coverage"  
  }  
}
```

json

改动[test/index.spec.ts](#)的代码，因为代码改动较大就不贴出来了。测试脚本会重新执行，输出以下信息表示测试通过，但覆盖率未完全通过。

```
node
G~ 4/08, 12:45 AM yangzw JowayYoungMac node • node • bash
PASS test/index.spec.ts
✓ Common Array GroupArrKey (4 ms)
✓ Common Array RecordArrPosition (1 ms)
✓ Common Array StatArrCount
✓ Common Array StatArrKeyword (1 ms)

-----|-----|-----|-----|-----|-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files |    100 |    76.19 |    100 |    100 |
array.ts  |    100 |    76.19 |    100 |    100 | 20,29,37,46
-----|-----|-----|-----|-----|-----
Jest: "global" coverage threshold for branches (100%) not met: 76.19%
Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        0.356 s
Ran all test suites.
```

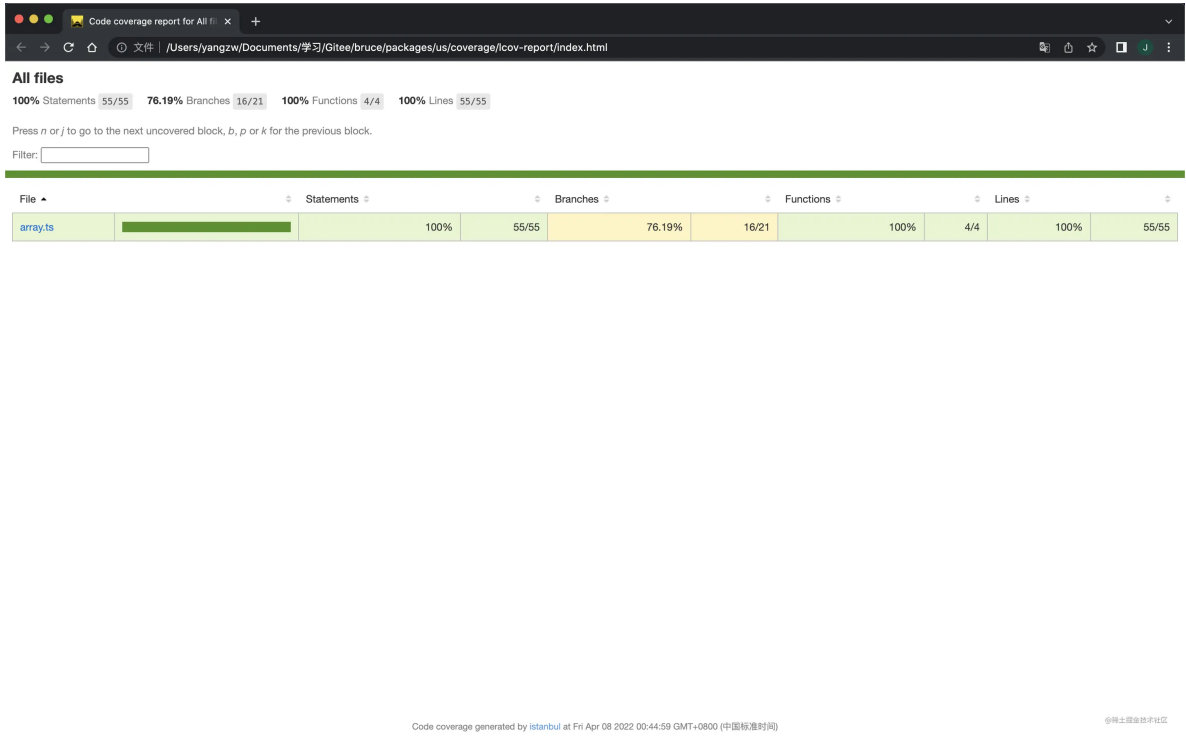
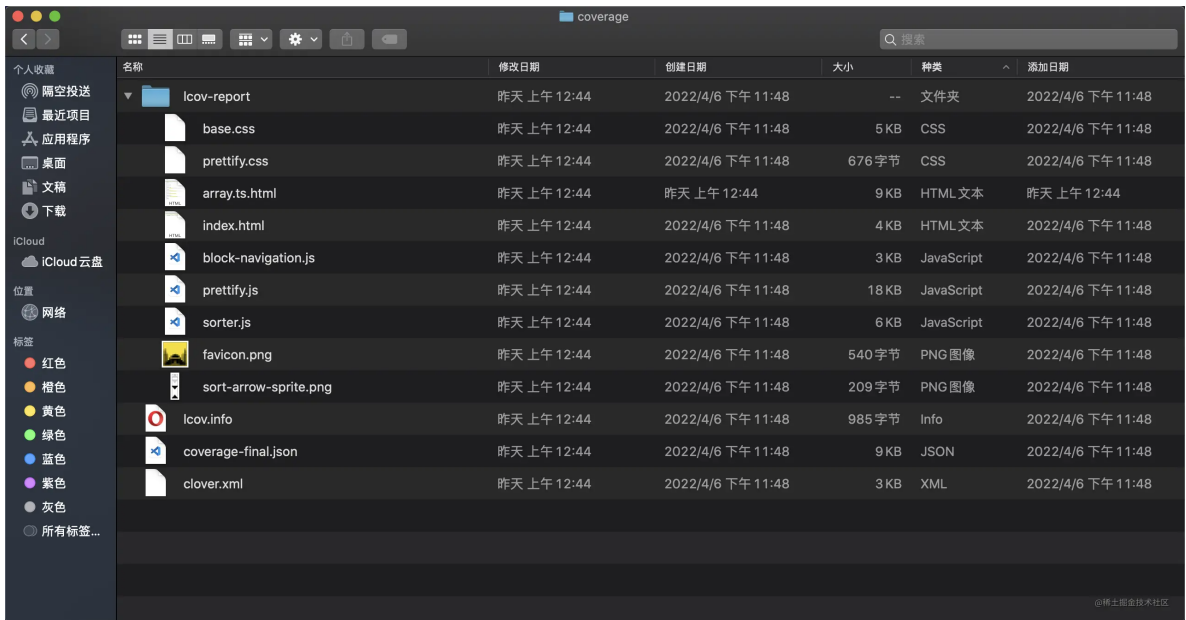
这是一个关于测试覆盖率的说明，从中可知四个测试用例都通过，但覆盖率只有 76.19%，不通过的部分集中在 Branch。那这些带有 % 的参数表示什么含义？

参数	说明	描述
%Stmts	语句覆盖率	是否每个语句都执行
%Branch	分支覆盖率	是否每个条件都执行
%Funcs	函数覆盖率	是否每个函数都调用
%Lines	行覆盖率	是否每行代码都执行

其中 %Stmts 与 %Lines 可能有些歧义，有缩写代码或压缩代码的情况下，行覆盖率的颗粒度可能大于 语句覆盖率，因为可允许一行代码包括多条语句。

```
function Sum(...vals: number[]): number {
    if (vals.length === 0) { return -1; } else if (vals.length === 1) { return 0; }
}
```

除了在控制台输出图表信息，还会生成一个 coverage 文件夹，点击 index.html 就会打开一个详细的测试报告，可根据测试报告的详细信息完善 单元测试 的细节。



使用：认识常见匹配器

上述代码都有用到 `expect()`、`test()` 和 `toBe()` 三个函数，它们组合起来表示一个测试用例中运行结果匹配期待结果，检验是否符合某种匹配状态。该匹配状态又称 **匹配器**，可能是 **值相等**、**值全等**、**满足范围值** 等。

`toBe()` 是一个使用频率很高的匹配器，除了它，`jest` 还提供一些很好用的匹配器。

toBe()

检查对象是否全等某值，类似 `===` 。

```
test("值是否相等3", () => {  
    expect(1 + 2).toBe(3); // 通过  
});
```

js

toBeLessThan()

检查对象是否小于某值，类似 `<` 。

```
test("值是否小于3", () => {  
    expect(1 + 2).toBeLessThan(3); // 不通过  
});
```

js

toBeGreaterThan()

检查对象是否大于某值，类似 `>` 。

```
test("值是否大于3", () => {  
    expect(1 + 2).toBeGreaterThan(3); // 不通过  
});
```

js

toBeLessThanOrEqual()

检查对象是否小于等于某值，类似 `<=` 。

```
test("值是否小于等于3", () => {  
    expect(1 + 2).toBeLessThanOrEqual(3); // 通过  
});
```

js

toBeGreaterThanOrEqual()

检查对象是否大于等于某值，类似 `>=` 。

js

```
test("值是否大于等于3", () => {  
    expect(1 + 2).toBeGreaterThanOrEqual(3); // 通过  
});
```

toBeCloseTo()

检查对象是否约等于某值，类似 $\approx$ 。

js

```
test("0.1+0.2是否约等于0.3", () => {  
    expect(0.1 + 0.2).toBe(0.3); // 不通过  
    expect(0.1 + 0.2).toBeCloseTo(0.3); // 通过  
});
```

toEqual()

测试两个对象的值是否相等，只对比值，不对比引用地址。该函数用在 引用类型 中更佳，例如 数组 与 对象 。

js

```
test("两数组的内容是否相等", () => {  
    const arr1 = [0, 1, 2];  
    const arr2 = [0, 1, 2];  
    expect(arr1).toEqual(arr2); // 通过  
});
```

toBeUndefined()

检查对象是否为 `undefined`。

js

```
test("值是否为undefined", () => {  
    const val = undefined;  
    expect(val).toBeUndefined(); // 通过  
});
```

toBeNull()

检查对象是否为 `null`。

js

```
test("值是否为null", () => {  
    const val = null;  
    expect(val).toBeNull(); // 通过  
});
```

toBeTruthy()

检查对象转换为布尔后是否为 **true** 。

js

```
test("转换值是否为true", () => {  
    expect(undefined).toBeTruthy(); // 不通过  
    expect(null).toBeTruthy(); // 不通过  
    expect("").toBeTruthy(); // 不通过  
    expect(0).toBeTruthy(); // 不通过  
    expect(false).toBeTruthy(); // 不通过  
});
```

toBeFalsy()

检查对象转换为布尔后是否为 **false** 。

js

```
test("转换值是否为false", () => {  
    expect(undefined).toBeFalsy(); // 通过  
    expect(null).toBeFalsy(); // 通过  
    expect("").toBeFalsy(); // 通过  
    expect(0).toBeFalsy(); // 通过  
    expect(false).toBeFalsy(); // 通过  
});
```

toMatch()

检查对象是否包括字符串或匹配正则，类似字符串的 **includes()** 与 **match()** 。

js

```
test("值是否被匹配", () => {  
    expect("https://yangzw.vip").toMatch("yangzw"); // 通过  
    expect("https://yangzw.vip").toMatch(/^https/); // 通过  
});
```

toContain()

检查对象是否被数组包括，类似数组的 `includes()`。

```
test("值是否被包括", () => {  
  const list = [0, 1, 2];  
  expect(list).toContain(1); // 通过  
});
```

js

基本上掌握上述函数就能应付很多需求，当然想了解更多期望函数，可查看[Jest预期函数](#)。

结语

单元测试在 前端工程化 中完善起来较吃力，毕竟编写测试用例需花很多时间，即使是大厂前端团队也会衡量该成本是否在预算范围内。对于一些 大型项目 或 开源项目，单元测试 还是很有必要接入的；对于一些小型项目或 KPI项目，就需考虑考虑了。

像组件库与工具库这些业务性较强的类库，还是建议花费一些时间对接上 单元测试，毕竟开发它们是基于 模块化 与 组件化，最终目的是应用到不同项目中，那对代码设计与代码实现肯定相比业务代码会更高，那 单元测试 就是改善代码质量与要求的最佳方案。

本章内容到此为止，希望能对你有所启发，欢迎你把自己的学习心得打到评论区！

☑ 示例项目：[fe-engineering](#)

☑ 正式项目：[bruce](#)

留言

输入评论（Enter换行，Ctrl + Enter发送）

发表评论

全部评论 (6)



qingkooo  前端开发 1月前

请问作者，这张jest和mocha的对比图是哪个网站提供的功能呢，麻烦告知下谢谢了
p9-juejin.byteimg.com



👍 点赞 💬 1



徐小佑  1月前

npmtrends

👍 点赞 💬 回复



Emma_  2月前

图片炸了

👍 点赞 💬 1



JowayYoung  (作者) 2月前

找下掘金，我也无能为力 😂

👍 点赞 💬 回复



小迷  前端 2月前

标记一下下

👍 点赞 💬 回复



EEEEEEEEEE   前端 3月前

打卡

👍 点赞 💬 回复